

CSCI544: Homework Assignment №3

Due on March 27, 2026

Introduction

This assignment gives you hands-on experience on building deep learning models on named entity recognition (NER). We will use the CoNLL-2003 corpus to build a neural network for NER. In the folder named *data*, there are three files: *train*, *dev* and *test*. In the files of *train* and *dev*, we provide you with the sentences with human-annotated NER tags. In the file of *test*, we provide only the raw sentences. The data format is that, each line contains three items separated by a *white space* symbol. The first item is the index of the word in the sentence, the second item is the word, and the third item is the corresponding NER tag. To clearly define the boundaries of named entities, the dataset utilizes the BIO (Begin, Inside, Outside) tagging scheme:

1. B- [type] (Beginning): This tag marks the very first token of a specific entity. Even if the entity is only one word long, it must start with a B-.
2. I- [type] (Inside): This tag is used for any subsequent tokens that belong to the same entity.
3. O (Outside): This tag is for any token that does not belong to any named entity (standard words, punctuation, etc.).

There will be a blank line at the end of one sentence. We also provide you with a file named *glove.6B.100d.gz*, which is the GloVe word embeddings [1].

We also provide the official evaluation script *conll03eval* to evaluate the results of the model. To use the script, you need to install *perl* and prepare your prediction file in the following format:

idx	word	pred	(1)
-----	------	------	-----

where there is a white space between two columns (in other words, this is the same format as the training data, but with predicted labels instead of the gold labels). *gold* is the gold-standard NER tag and *pred* is the model-predicted tag. Then execute the command line:

```
python eval.py -p {predicted file} -g {gold-standard file}
```

where *{predicted file}* is the prediction file in the prepared format.

Note: We have made this homework open-ended by design to emulate the real-world process of developing NLP systems. You will have to do some amount of iteration yourself to identify good hyper-parameters (both tasks) and modeling strategies for capitalization (Task 2). There is no single right answer to these questions; you will be graded on your model's performance on the dev and test data, as well as your ability to explain your solution in your write-up. If you're having a hard time iterating on your strategy because training your model is taking a very long time, make sure you are implementing batching correctly (see slides pages 33-36 in csci544-PyTorch-tutorial (Week 4) for implementation of batching in Pytorch).

Submission details

You are **required** to implement your solution in **Pytorch**; other frameworks (e.g., TensorFlow, Keras) are not permitted. For reproducibility, you must use random state = 42 whenever applicable (e.g., sampling data, model initialization, etc.).

Please follow the instructions and submit a zipped folder containing:

1. A model file named *blstm1.pt* for the trained model in Task 1.
2. A model file named *blstm2.pt* for the trained model in Task 2.
3. Predictions of both dev and test data from Task 1 and Task 2. Name the file with *dev1.out*, *dev2.out*, *test1.out* and *test2.out*, respectively. All these files should be in the same format of training data.
4. You must also submit your **Python code** and a **README file** describing how to run the code to generate the prediction files. In the README, please provide the command line for training and the command line for generating the prediction files. We will use these commands to reproduce your reported results on the development set.

5. A **PDF file** which contains answers to the questions in the assignment along with a clear description about your solution, including **all** the hyper-parameters used in network architecture and model training.

Task 1: Simple Bidirectional LSTM model (40 points)

The first task is to build a simple bidirectional LSTM model (see slides page 39 in csci-544-sequence-labeling (Week 3) for the network architecture) for NER.

Task. Implementing the bidirectional LSTM network with PyTorch. The architecture of the network is:

Embedding $\rightarrow$ BLSTM $\rightarrow$ Linear $\rightarrow$ ELU $\rightarrow$ classifier

Note that we are using the ELU non-linearity, which is similar to ReLU but is differentiable everywhere.¹ Use the following hyper-parameters in your network:

embedding dim	100
number of LSTM layers	1
LSTM hidden dim	256
LSTM Dropout	0.33
Linear output dim	128

Train this simple BLSTM model with the training data on NER with SGD as the optimizer. Please tune other hyper-parameters that are not specified in the above table, such as number of epochs, batch size, learning rate, and learning rate scheduling. Predict the NER tags of the sentences in the dev and test data and output the predictions to files named *dev1.out* and *test1.out*, respectively, using the same format of training data. What are the precision, recall and F1 score on the dev data? (Hint: a reasonable F1 score on dev is 77%).

¹See <https://docs.pytorch.org/docs/stable/generated/torch.nn.ELU.html> for details.

Task 2: Using GloVe word embeddings (60 points)

The second task is to use the provided GloVe word embeddings to improve the BLSTM in Task 1. The way we use the GloVe word embeddings is straightforward: we initialize the embeddings in our neural network with the corresponding vectors in GloVe.

Note that GloVe is case-insensitive, but our NER model should be case-sensitive because capitalization is an important information for NER. You should develop your own strategy for making your model case sensitive and describe this strategy in your write-up. Similar to Task1, predict the NER tags of the sentences in the dev and test data and output the predictions to files named *dev2.out* and *test2.out*, respectively, using the same format of training data.

What are the precision, recall and F1 score on the dev data? (Hint: a reasonable F1 score on dev is 88%.

Bonus: LSTM-CNN model (10 + 10 points)

The bonus task is to equip the BLSTM model in Task 2 with a CNN module to capture character-level information (see slides page 42 in csci-544-sequence-labeling (Week 3) for the network architecture). The character embedding dimension is set to 30. You need to tune other hyper-parameters of CNN module, such as the number of CNN layers, the kernel size and output dimension of each CNN layer. What are the precision, recall and F1 score on the dev data? Predict the NER tags of the sentences in the test data and output the predictions in a file named *pred*, in the same format of training data. You may receive up to 10 bonus points for the absolute performance of your model, and an additional 10 bonus points if your submission ranks in the top 5 in the class.

References

- [1] Jeffrey Pennington, Richard Socher, and Christopher D Manning. Glove: Global vectors for word representation. In *Proceedings of the 2014 con-*

ference on empirical methods in natural language processing (EMNLP),
pages 1532–1543, 2014.